

A PROJECT REPORT ON
CONVERT NATURAL LANGUAGE INTO DATABASE CONFIGURATION JSON

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)

SUBMITTED BY

SARTHIK BANGROO
DISHA SAXENA
NIKHIL SHRIRAO

Exam No :B400050581
Exam No :B400050585
Exam No :B400050599



DEPARTMENT OF COMPUTER ENGINEERING

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

DHANKAWADI, PUNE 411043

SAVITRIBAI PHULE PUNE UNIVERSITY

2024-2025



CERTIFICATE

This is to certify that the project report entitled

“CONVERT NATURAL LANGUAGE INTO DATABASE CONFIGURATION JSON ”

Submitted by

SARTHIK BANGROO

Exam No : B400050581

DISHA SAXENA

Exam No : B400050585

NIKHIL SHRIRAO

Exam No : B400050599

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. Rutuja Kulkarni** and it is approved for the fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Prof. Rutuja Kulkarni
Guide,
Department of Computer Engineering

Dr. G. V. Kale
Head,
Department of Computer Engineering

Dr. S. T. Gandhe
Principal,
Pune Institute of Computer Technology

Place : Pune
Date :

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to **Prof. Rutuja Kulkarni**, our internal project guide, for her constant support, invaluable guidance, and encouragement throughout the duration of this project. Her deep subject knowledge, timely feedback, and mentorship played a pivotal role in shaping our approach and execution.

We are equally grateful to **Dr. G. V. Kale**, Head of the Department of Computer Engineering, for fostering a research-oriented environment and providing us with the infrastructure and academic resources necessary to carry out our work effectively. His support and motivation have greatly inspired us to pursue excellence in our academic endeavors.

Our sincere thanks also go to **Dr. S. T. Gandhe**, Principal of PICT, for his continuous encouragement and for facilitating an environment that supports innovation and collaboration between academia and industry.

We would especially like to acknowledge **Mr. Swapnil Bhusari**, our external mentor from **Veritas Technologies**, whose expertise, professional insights, and practical suggestions significantly enriched our understanding and helped us align our academic knowledge with real-world applications. His mentorship enabled us to overcome critical challenges and guided us toward developing a more robust and effective solution.

We are truly thankful to **Veritas Technologies** for sponsoring this project and for providing us with an exceptional opportunity to gain hands-on experience with industry practices. Their support extended beyond technical resources and offered us a glimpse into professional project development cycles and expectations.

Lastly, we extend our appreciation to both **PICT** and **Veritas Technologies** for their collaborative support, which made this project not only possible but also a deeply enriching experience. The learnings and experiences gained through this endeavor will undoubtedly contribute to our academic growth and professional journey.

SARTHIK BANGROO,
DISHA SAXENA,
NIKHIL SHRIRAO.
NAME OF THE STUDENTS

ABSTRACT

The goal of a natural language interface for a database configuration is to transform human descriptions of data requirements into precise specifications. In this paper, we focus on a system which translates natural language into structured JSON configurations which includes database schemas and queries in the form of JSON objects. We employ Mistral's Large Language Model (LLM) for this task. It integrates numerous techniques such as prompt framing, output constraints, validation, and checking the bounds of the compliant JSON to ensure that the generated JSON is syntactically correct and aligned with the user's intents. We outline how Mistral AI's large language model was used to parse the semantics of user instructions from natural language into structured representations using a customized approach to prompt construction, responsive input methods, and decentralized output validation.

The accompanying system architecture is presented together with illustrative examples, including translating English specifications of a table schema into JSON-formatted definitions (solving the table description problem). We determine the effectiveness of the approach using both expert-driven samples and formal evaluations based on set criteria assessing the validity of the JSON document and factual correctness of its content. The findings illustrate that appropriate system design enables the use of a 7 billion parameter Mistral model to generate complex JSON database configurations expressed in natural language with reasonable accuracy. We have automated processes for quick and agile data engineering prototypes, which we expand and refine as future work by smoothing system edges such as adaptive learning.

TABLE OF CONTENTS

LIST OF FIGURES

i

LIST OF TABLES

ii

Sr. No.	Title of Chapter	Page No.
01	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Project Scope & Limitations	4
1.5	Methodologies of Problem solving	4
02	Literature Survey	5
03	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	10
3.2.1	Natural Language To Json Translation	10
3.2.2	Real-Time Interaction Using Websockets	10
3.2.3	Schema-Aware Prompting	10
3.3	External Interface Requirements	11
3.3.1	User Interfaces	11
3.3.2	Hardware Interfaces	11
3.3.3	Software Interfaces	11
3.3.4	Communication Interfaces	11
3.4	Nonfunctional Requirements	11
3.4.1	Performance Requirements	11
3.4.2	Safety Requirements	11
3.4.3	Security Requirements	12
3.4.4	Software Quality Attributes	12
3.5	System Requirements	12
3.5.1	Database Requirements	12
3.5.2	Software Requirements	12
3.5.3	Hardware Requirements	13
3.6	Analysis Models: SDLC Model to be applied	14
04	System Design	15
4.1	System Architecture	16
4.2	UML Diagrams	17
05	Project Plan	20
5.1	Project Estimate	21
5.1.1	Reconciled Estimates	21
5.2	Risk Management	22
5.2.1	Risk Identification	22
5.2.2	Risk Analysis	23
5.2.3	Overview of Risk Mitigation, Monitoring, Management	24

5.3	Project Schedule	25
5.3.1	Project Taskset	25
5.3.1	Task Network	26
5.3.3	Timeline Chart	27
5.4	Team Organization	28
5.4.1	Team structure	28
5.4.2	Management reporting and communication	29
06	Project Implementation	30
6.1	Overview of Project Modules	31
6.2	Tools and Technologies Used	32
6.3	Algorithm Details	33
07	Software Testing	35
7.1	Type of Testing	36
7.2	Test cases & Test Results	36
08	Results	37
8.1	Outcomes	38
8.2	Screen Shots	39
09	Conclusions	47
9.1	Conclusions	48
9.2	Future Work	48
9.3	Applications	49
	Appendix A: Problem statement feasibility assessment using, satisfiability analysis and NP Hard, NP-Complete or P type using modern algebra and relevant mathematical models.	50
	Appendix B: Details of paper publication: name of the conference/journal, comments of reviewers, certificate, paper. And achievements if any	
	Appendix C: Plagiarism Report of project report.	
	References	55
	Thomas Noltey, Hans Hansson, Lucia Lo Belloz, "Communication Buses for Automotive Applications" In <i>Proceedings of the 3rd Information Survivability Workshop (ISW-2007)</i> , Boston, Massachusetts, USA, October 2007. IEEE Computer Society.	

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE No.
3.6	Agile SDLC Model	15
4.1	System Architecture	17
4.2.1	Class Diagram	18
4.2.2	State Diagram	19
4.2.3	Use Case Diagram	20
5.3.2	Task Network	27
5.3.3	Timeline Chart	28
8.2.1	Result Screenshot 01	39
8.2.2	Result Screenshot 02	40
8.2.3	Result Screenshot 03	41
8.2.4	Result Screenshot 04	42
8.2.5	Result Screenshot 05	43
8.2.6	Result Screenshot 06	44
8.2.7	Result Screenshot 07	45
8.2.8	Result Screenshot 08	46

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE No.
2.1	Literature Survey	06
5.1.1	Time Estimates	22
5.1.2	Reconciled Estimates	22
5.2.2	Risk Analysis	24
7.2	Test Cases and Results	38
A.1	Problem Statement Feasibility Assessment	48

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In recent years, the barrier between natural language and structured database operations has gradually decreased, thanks to advancements in Natural Language Processing (NLP) and Large Language Models (LLMs) [7][8]. This project focuses on bridging the gap between non-technical users and backend systems by allowing natural language descriptions to be converted into database configuration JSON [3][2]. The core idea is to eliminate the need for manually writing SQL queries or understanding database schema structures—users can simply describe their data needs in plain English [6][4].

To achieve this, we designed a full-stack application that leverages LLMs to parse user input and produce a well-structured JSON configuration that represents SQL-like queries and operations [11]. This JSON includes fields such as query type, tables, columns, joins, conditions, and more. Instead of fine-tuning the model, the system utilizes prompt engineering techniques to extract accurate and consistent outputs [12][1]. The backend is built using Flask and WebSockets to provide real-time feedback, while the frontend offers an intuitive interface for users to input their queries.

This solution demonstrates how carefully crafted prompts, structured schema parsing, and lightweight architecture can work together to simplify complex technical workflows. The project provides a foundation for further expansion into auto-query generation, database design, and low-code tools for data professionals and beginners alike [9][13][16].

1.2 MOTIVATION

The motivation for this project stems from the increasing demand for intuitive data interaction methods in enterprise environments. Most database systems today require users to have a working knowledge of query languages like SQL, limiting accessibility to non-technical users. The goal of this project is to bridge the gap between users and databases by leveraging LLMs, allowing users to retrieve data without prior knowledge of database structures or query languages.

Another key motivator is the dynamic nature of databases in modern applications. As database schemas evolve, it becomes necessary to have a system that can adapt and generate queries that are up to date with schema changes. Current static systems fail to offer this level of adaptability, leading to the need for dynamic query generation powered by LLMs.

This project, therefore, aims to simplify data access, improve flexibility, and provide a more intuitive interface between end-users and complex database systems.

1.3 PROBLEM DEFINITION AND OBJECTIVES

1.3.1 Problem Definition

Interacting with relational databases typically requires knowledge of SQL syntax and an understanding of underlying schema structures. For non-technical users or even developers dealing with complex queries, this becomes a barrier to efficient data retrieval and manipulation. There is a clear need for a user-friendly system that can interpret natural language inputs and translate them into a machine-readable query format.

1.3.2 Objectives

- To develop a system that converts natural language descriptions into structured JSON for SQL query generation.
- To utilize a large language model (LLM) with prompt engineering to interpret intent accurately.
- To design a flexible schema that captures essential components of database queries (e.g., query type, columns, joins, conditions).
- To implement a real-time web interface for input and output interaction using Flask and WebSockets.
- To ensure the output JSON is standardized, complete, and compatible with downstream query generation or automation tools.

1.4 PROJECT SCOPE & LIMITATIONS

1.4.1 Project Scope

The system supports most common SQL operations like SELECT, INSERT, UPDATE etc.

- JSON output includes key fields such as query_type, tables, columns, conditions, group_by, and more.
- Real-time communication is implemented using Flask-SocketIO.
- The frontend allows users to input queries, view responses, and check errors, making it accessible and interactive.

1.4.2 Limitations:

- The system does not directly execute SQL queries—only generates JSON representations.
- Complex or ambiguous natural language inputs may lead to inconsistent interpretations.
- The quality of output is highly dependent on prompt clarity and the LLM's performance.
- It assumes users reference existing schema elements correctly (e.g. table or column names).
- No persistent memory or context tracking between multiple queries is implemented.

1.5 METHODOLOGIES OF PROBLEM SOLVING

- **Requirement Analysis:** Identified the need for translating natural language into structured data formats for databases.
- **Prompt Engineering:** Created structured prompts that guide the LLM in generating consistent and complete JSON outputs.
- **Schema Definition:** Used LangChain's StructuredOutputParser with a predefined response schema to ensure accuracy.
- **Real-time Communication:** Integrated Flask with SocketIO for seamless real-time data exchange between frontend and backend.

CHAPTER 2

LITERATURE SURVEY

The advancement of Natural Language Interfaces (NLIs) for querying databases has been an active area of research [3][4][6], with significant strides made in both SQL and NoSQL systems [2][14]. This literature survey provides a comprehensive analysis of the most relevant frameworks, methods, and approaches that have been proposed for converting natural language into database queries [3][15], including both traditional methods and emerging AI-driven techniques [7][8][9].

Table 2.1 Project Literature Survey

Paper Title	Authors/Year	Focus	Key Contributions	Challenges
Frameworks for Querying Databases Using Natural Language: A Literature Review	Hafsa Shareef Dar, M. Ikramullah Lali, Moin Ul Din, Khalid Mahmood Malik, Syed Ahmad Chan Bukhari / 2021	Overview of 47 frameworks for converting natural language into SQL/NoSQL queries.	-Categorized frameworks into statistical, symbolic, connectionist, semantic matching, and pattern matching. - Most frameworks focus on SQL; 70% of work in this area supports SQL.	-accurately translating ambiguous or complex natural language queries into structured database queries, ensuring both syntactic and semantic correctness.
Efficiency of JSON approach for Data Extraction and Query Retrieval	Universiti Sultan Zainal Abidin (UniSZA) / 2021	Compares XML and JSON approaches for data extraction and query retrieval in large databases.	- Found that JSON provides better storage and query speed compared to XML. - Identifies JSON as more suitable for growing datasets.	-performance degradation when handling complex, deeply nested structures or very large datasets, leading to slower query execution times.
How To Convert Natural Language to SQL Queries	Google's Big Query / 2023	Discusses how to convert natural language into SQL queries using Data QnA interface in Big Query.	- Explores encoder-decoder models for converting text to SQL. - Real-time query generation via	- The Data QnA interface is tightly integrated with Google's ecosystem, limiting its applicability to other platforms or custom

			Google's trained models is shown.	databases.
Deep Learning Driven Natural Language Text to SQL Query Conversion	International Conference on Computational Linguistics (COLING) / 2021	Focuses on using deep learning for text-to-SQL translation across different domains.	- Covers various datasets like ATIS and Spider. - Discusses encoder-decoder frameworks for improving query accuracy and handling complex queries.	-While encoder-decoder frameworks improve query accuracy, handling highly complex queries across different domains remains a challenge.
A Comparative Survey of Recent Natural Language Interfaces for Databases	VLDB Journal (Springer) / 2021	Surveys (NLIs) for databases, focusing on SQL/SPARQL systems.	- Analyzes performance of NLIs in handling complex queries. - Provides insights into scalability.	-Difficulty in handling ambiguous or context-dependent queries, which can result in inaccurate or incomplete query translations.
A Review on Natural Language Interface to Databases (NLIDB)	Swathi Ramesh, Sharada Ramaswamy / 2020	Reviews different approaches to natural language interfaces for databases.	- Highlights rule-based and machine learning-based methods for NLIDB. - Emphasizes the need for schema-awareness in NLI systems.	-The review highlights that rule-based systems are inflexible, while machine learning-based methods require improved schema-awareness to enhance accuracy.
An Empirical Study on Text-to-SQL Conversion using Large Language Models	OpenAI / 2022	Evaluates the performance of GPT-3 for converting natural language queries to SQL.	- Demonstrates the ability of large language models to handle ambiguous prompts and generate accurate SQL queries. - Highlights challenges in handling schema	-It struggles with handling schema variations across different databases.

			variations.	
Natural Language Interface for Relational Databases: Challenges and Future Directions	Vinodkumar Ekbote, Swarnalatha P / 2021	Examines the integration of natural language interfaces with relational databases.	- Highlights issues in handling ambiguous queries and managing large-scale database systems. - Discusses the importance of NLP models' adaptation to specific database schemas.	-The paper points out challenges in managing large-scale database systems and handling ambiguous queries, with an emphasis on the need for better adaptation of NLP models to specific database schemas.
Towards Semantic Parsing: Querying Databases Using Neural Approaches	Xi Victoria Lin, Richard Socher / 2020	Discusses neural approaches for semantic parsing in querying databases using natural language.	- Introduces techniques for neural semantic parsing, enabling better contextual understanding in query generation. - Emphasizes real-time response generation and query optimization.	-While the paper introduces promising techniques for neural semantic parsing, it notes that real-time response generation and query optimization still face challenges in achieving consistency.
Natural Language to SQL: State of the Art and Open Research Issues	Shivani Agarwal, Aditi Patil / 2022	Surveys recent methods for text-to-SQL generation, focusing on domain-specific challenges.	- Presents advances in domain adaptation and schema understanding. - Highlights key challenges such as schema dependency and generalization across different domains.	-The survey highlights key challenges, including schema dependency and the difficulty in generalizing models across different domains, limiting the adaptability of text-to-SQL systems.

CHAPTER 03

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 ASSUMPTIONS AND DEPENDENCIES

This system assumes the presence of a pre-trained large language model (LLM) like LLaMA or GPT, integrated using LangChain. It also assumes access to a database system that accepts structured JSON queries. Dependencies include Flask for the backend, WebSockets for real-time communication, and LangChain for prompt engineering and output parsing.

3.2 FUNCTIONAL REQUIREMENTS

3.2.1 Natural Language To Json Translation

The system shall accept user input in the form of natural language and convert it into a valid structured JSON query that aligns with the backend database schema.

- **Inputs:** Natural language queries from the user.
- **Outputs:** JSON object representing the query.
- **Functionality:** Use LLM via LangChain to interpret and convert inputs using a defined schema and structured prompt.

3.2.2 Real-Time Interaction Using Websockets

The system shall use WebSockets to allow real-time interaction between the frontend and backend, making the querying experience seamless and dynamic.

- **Inputs:** WebSocket messages containing user input.
- **Outputs:** WebSocket responses containing structured JSON or database results.
- **Functionality:** Establish bi-directional communication for continuous prompt–response flow.

3.2.3 Schema-Aware Prompting

The system shall utilize schema-aware prompts to ensure generated queries are syntactically and semantically correct.

- **Inputs:** Schema definitions and user prompt.
- **Outputs:** Accurate JSON query structure.
- **Functionality:** Include schema in prompt context to guide LLM response formatting.

3.3 EXTERNAL INTERFACE REQUIREMENTS

3.3.1 User Interfaces

The web application shall provide a minimal and interactive frontend to enter natural language queries and view structured JSON or results in real time.

3.3.2 Hardware Interfaces

There are no specific hardware interface requirements aside from a standard computing environment that can support Python-based applications.

3.3.3 Software Interfaces

The system shall interface with the following software components:

- Flask Web Framework
- WebSockets Library (e.g., Flask-SocketIO)
- LangChain SDK
- LLM API or locally hosted model
- JSON parser and validator

3.3.4 Communication Interfaces

The system shall use WebSocket protocol for real-time communication between the frontend and backend. HTTP will be used for any fallback or REST-based operations.

3.4 NONFUNCTIONAL REQUIREMENTS

3.4.1 Performance Requirements

The system shall generate JSON responses within 2 seconds for standard user queries under typical load conditions.

3.4.2 Safety Requirements

No physical safety issues apply. However, care must be taken to handle erroneous queries gracefully without crashing the system.

3.4.3 Security Requirements

The system shall ensure data privacy and avoid exposing sensitive schema details to unauthorized users. Input sanitization and rate-limiting shall be implemented to prevent prompt injection or DoS attacks.

3.4.4 Software Quality Attributes

- **Reliability:** System should be resilient to input noise and network delays.
- **Maintainability:** Modular code design using Flask and LangChain.
- **Usability:** User-friendly interface with instant feedback.
- **Portability:** Platform-independent and can run on any system supporting Python 3.x.

3.5 SYSTEM REQUIREMENTS

3.5.1 Database Requirements

- Type: Relational Database Management System (RDBMS)
- Suggested DBMS: MySQL / PostgreSQL / SQLite (based on project scale)
- Features Required
 - Support for SQL queries
 - ACID compliance
 - Data backup and recovery options
 - User authentication and access control

3.5.2 Software Requirements

- Operating System: Windows 10 or above / Linux Ubuntu 20.04+ / macOS Monterey or later
- Development Tools:
 - Code Editor: VS Code / IntelliJ / PyCharm
 - Version Control: Git with GitHub/GitLab
 - Runtime: Java 11+ / Python 3.10+ / Node.js 18+ (depending on tech stack)
- Libraries & Frameworks:
 - Backend: Django / Express / Spring Boot

- Frontend: React / Angular / Vue
- API Integration: REST/GraphQL support

3.5.3 Hardware Requirements

- Minimum System Requirements:
 - Processor: Intel i5 8th Gen or equivalent
 - RAM: 8 GB
 - Storage: 256 GB SSD
 - Internet: Stable connection for development and testing
- Recommended System Requirements:
 - Processor: Intel i7 / AMD Ryzen 7 or higher
 - RAM: 16 GB or more
 - Storage: 512 GB SSD or higher

3.6 ANALYSIS MODELS: SDLC MODEL TO BE APPLIED

The **Agile Model** of Software Development Life Cycle (SDLC) is adopted for this project. Agile promotes adaptive planning, evolutionary development, early delivery, and continual improvement. It encourages rapid and flexible response to change, which is ideal for integrating evolving technologies like large language models (LLMs). Continuous feedback from testing the prompt outputs and user interactions will be incorporated into each sprint cycle to refine model behavior, improve performance, and ensure optimal usability.

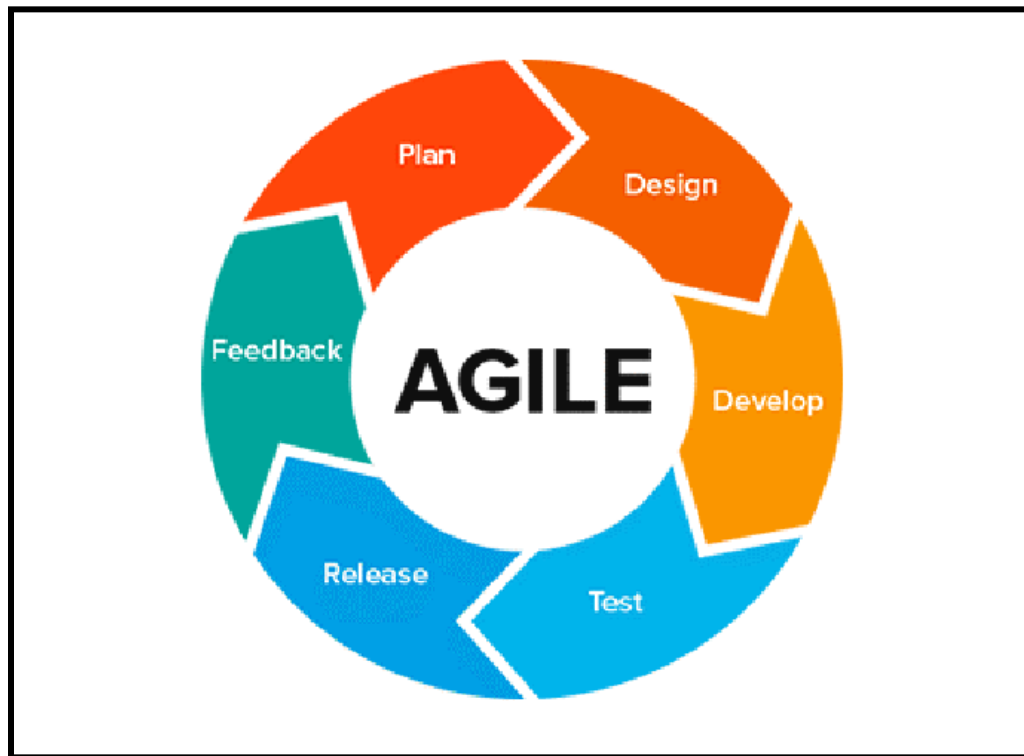


Fig No. 3.6 Agile SDLC Model [5]

CHAPTER 04

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

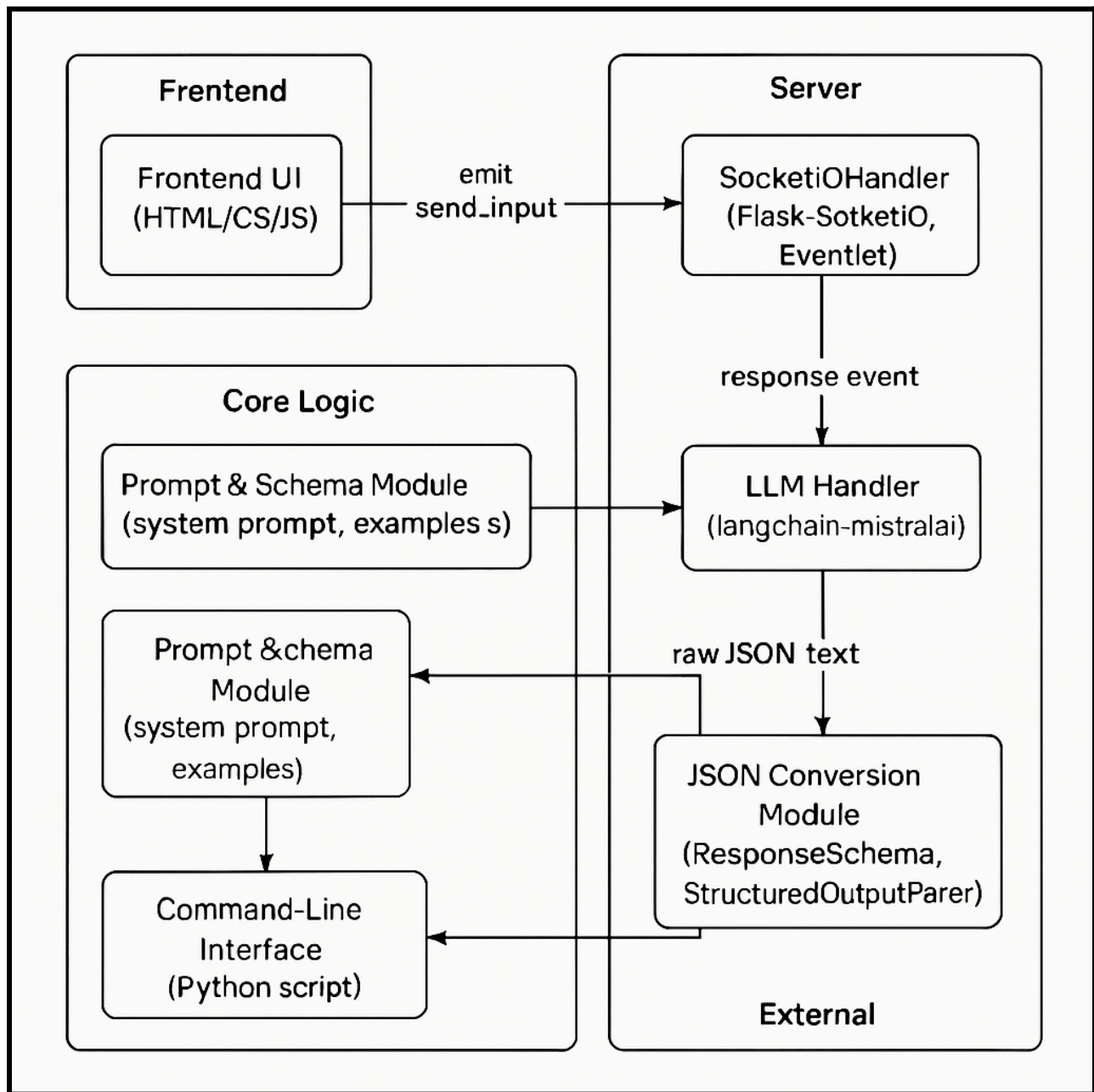


Fig no. 4.1 System Architecture

4.2 UML DIAGRAMS

4.2.1 Class Diagram

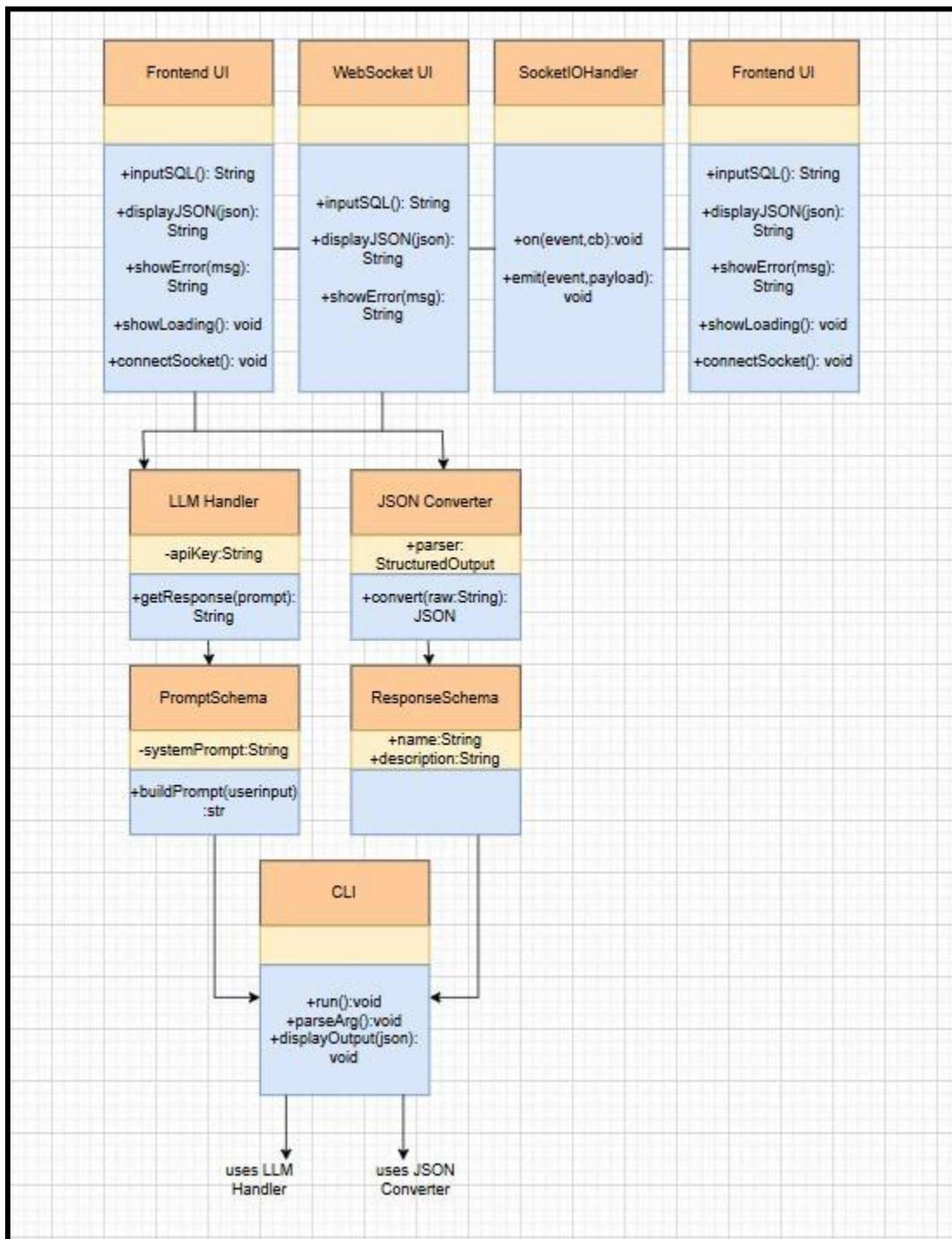


Fig 4.2.1 Class Diagram

4.2.2 State Diagram

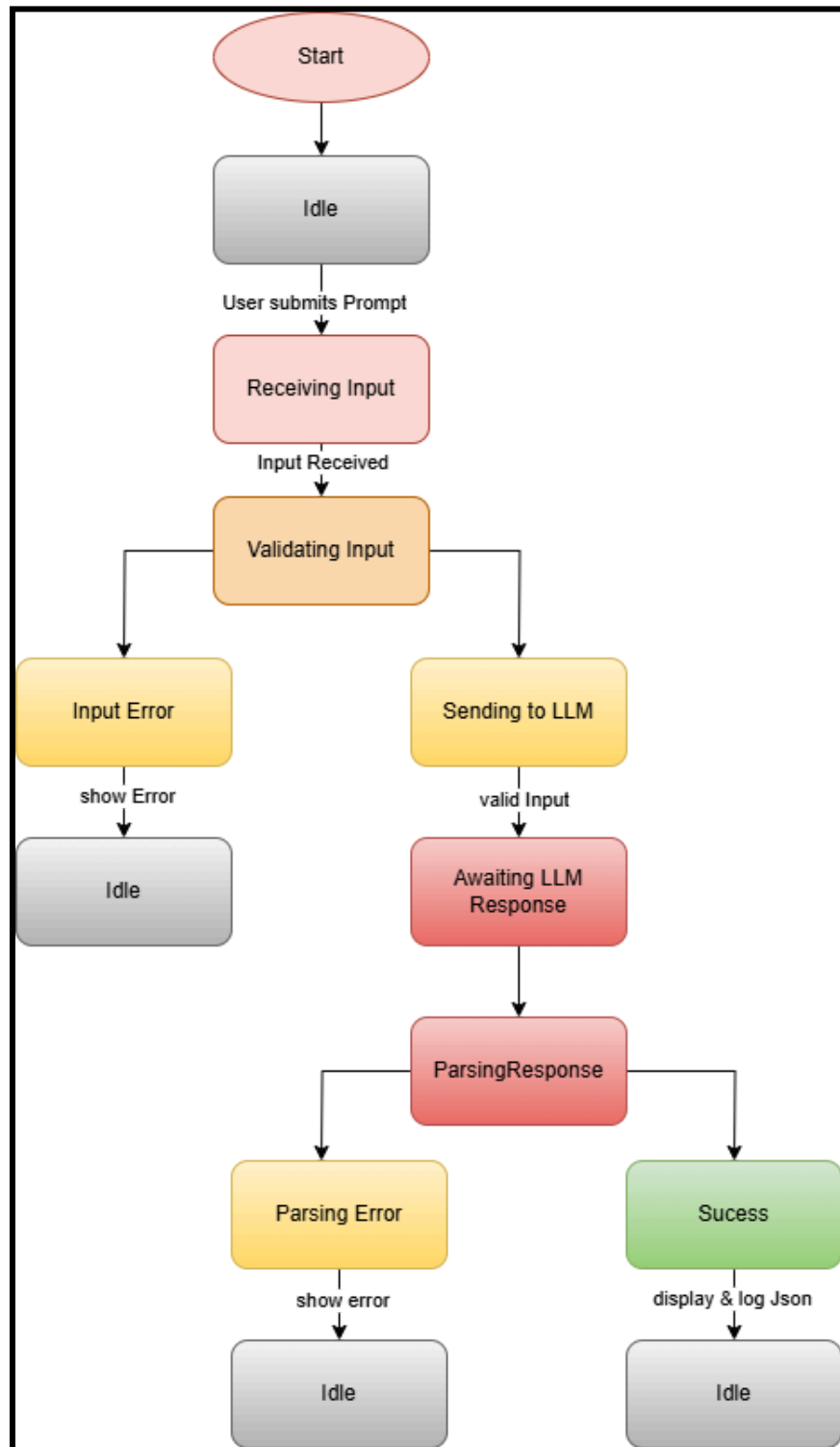


Fig 4.2.2 State Diagram

4.2.3 Use Case Diagram

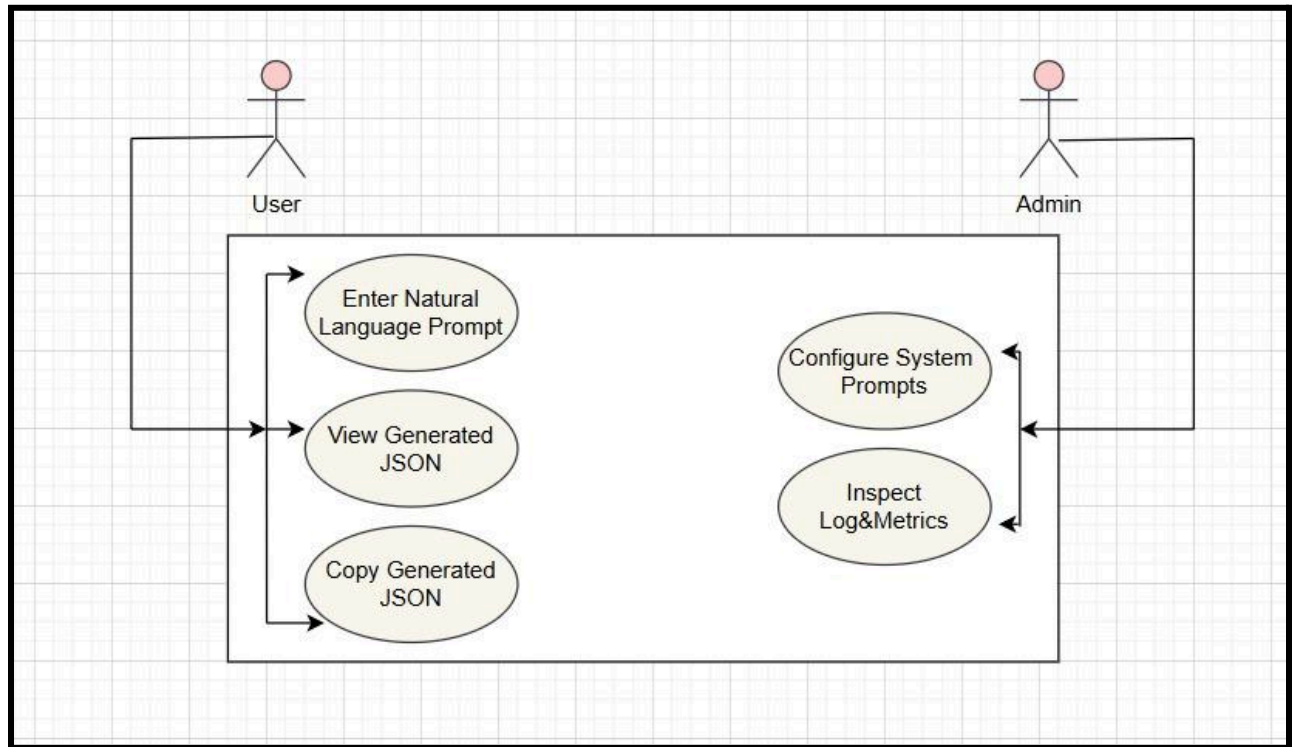


Fig 4.2.3 Use Case Diagram

CHAPTER 05

PROJECT PLAN

5.1 ESTIMATES

5.1.1 Time Estimates

Table 5.1.1 Time Estimates

Task	Estimated Duration
Requirements Gathering	6 weeks
Database Schema Design (PostgreSQL)	6 weeks
LLM Model Research and Configuration	6 weeks
NLP to JSON Conversion Module	6 weeks
Frontend Development	3 weeks
Backend Development (Integration)	7 weeks
Testing and Debugging	6 weeks
Final Integration and Deployment	3 weeks
Documentation and Report Writing	3 weeks
Total Estimated Duration	11 months

5.1.2 Reconciled Estimates

Table 5.1.2 Reconciled Estimates

Task	Initial Estimate	Reconciled Time
Requirements gathering and analysis	6 weeks	6 weeks
Database schema design	6 weeks	6 weeks
LLM Research and Configuration	6 weeks	6 weeks
NLP to JSON conversion module	6 weeks	7 weeks

Frontend development	3 week	3 week
Backend development (Integration)	7 weeks	7 weeks
Testing and debugging	6 weeks	5 weeks
Final integration and deployment	3 weeks	3 weeks
Documentation and report writing	3 weeks	2 weeks
Total	11 months	11 months

5.2 RISK MANAGEMENT

5.2.1 Risk Identification

- **Integration Issues:** Challenges in integrating the natural language processing (NLP) model with the database schema, especially when handling complex user queries or ensuring compatibility between the configuration and output.
- **Model Accuracy:** The LLM (Mistral) might fail to accurately map natural language prompts to JSON configurations, resulting in incorrect query generation.
- **Performance Bottlenecks:** Since the system will need to dynamically generate queries and interact with a database, performance issues like slow query processing or delays in real-time responses may arise.
- **Data Security:** If sensitive data is being handled, there may be risks of data breaches or unauthorized access, especially during the interaction between the LLM and the database.
- **Schema Changes:** In case the database schema evolves over time, the LLM may generate outdated or incorrect queries if the configuration is not updated.

5.2.2 Risk Analysis

Table 5.2.2 Risk Analysis

Risk	Likelihood	Impact	Description	Mitigation Strategy
Integration Issues	High	High	Issues with integrating the LLM and database, causing malfunction.	Comprehensive testing, debugging protocols, and regular system evaluations.
Model Accuracy	Medium	High	Inaccurate query generation can affect system reliability and user trust.	Regularly fine-tune the LLM using real-world data and test cases to improve precision.
Performance Bottlenecks	Medium	Medium	Delays in query execution might degrade user experience for large/complex queries.	Optimize using caching, load balancing, and monitoring tools to maintain performance.
Data Security	Low	High	Breach of sensitive data may lead to severe consequences.	Use HTTPS, data encryption, and regular security updates to safeguard info.
Schema Changes	Medium	Medium	Changes in the database schema may affect query generation accuracy.	Implement alerts for schema updates and dynamically adjust configurations to reflect changes.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Integration Issues:** Use thorough testing with different user prompts and configurations to ensure the NLP model and database are well-integrated. Establish protocols for debugging and resolving integration failures quickly.
- **Model Accuracy:** Implement multiple test cases for validating the accuracy of generated queries. Regularly fine-tune the model using real user queries and feedback to improve precision.
- **Performance Bottlenecks:** Optimize the system architecture by caching frequently used queries, scaling the database, and monitoring the system's performance using profiling tools.
- **Data Security:** Implement secure communication protocols such as HTTPS and encrypt sensitive data both in transit and at rest. Regularly update the system to handle new security vulnerabilities.
- **Schema Changes:** Set up automated alerts for database schema changes, and develop a system to dynamically update the JSON configuration and model parameters when schema modifications occur. Keep documentation on schema evolution to avoid discrepancies.

5.3 PROJECT SCHEDULE

5.3.1 Project Task Set

- **Requirements Gathering:** Define system requirements, identify user expectations, and document use cases for the natural language queries.
- **Model Training and Testing:** Train and test the LLM to interpret user prompts accurately and generate corresponding JSON configurations.
- **Database Schema Design:** Design a database schema to store relevant information and enable querying based on dynamic user inputs.
- **Development of JSON Configuration Generation:** Develop a mechanism that converts user prompts into structured JSON configurations for database querying.
- **User Interface Design:** Create a simple interface for users to input natural language queries and view results.
- **Integration and Testing:** Integrate all components and perform end-to-end testing to ensure proper functionality of the system.
- **Deployment:** Deploy the system, ensure it functions in the production environment, and provide necessary user documentation.

5.3.2 Task Network

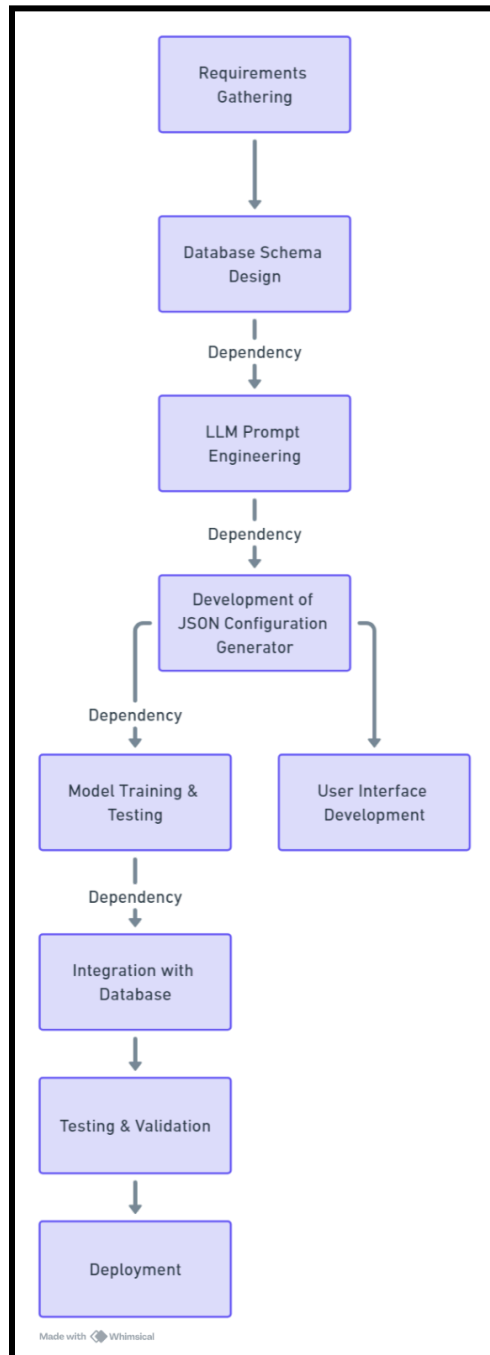


Fig 5.3.2 Task Network

5.3.3 Timeline Chart

Task/Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sept	Oct	Nov	Dec
Phase1							Research&Scope Definition					
Phase2									Tool Setup & Dataset Collection			
Phase3											LLM Integeration & Testing	
Phase4	Finalization, Optimization & Deployment											
Phase5					Report Submission and Final Review							

Fig 5.3.3 Timeline Chart

5.4 TEAM ORGANIZATION

5.4.1 Team Structure

Our project team consists of three members, each with specialized responsibilities:

1. **Sarthik** – Full Stack Developer with expertise in researching various Large Language Models (LLMs) and NLP techniques.

Role: Sarthik leads the exploration and selection of appropriate LLMs for the project, ensuring that the LLM meets the requirements for converting natural language queries into database actions. He also supports full stack development, contributing to the integration of the chosen LLM as well as play a key role in the implementation of the project.

2. **Disha** – Frontend Developer with a strong focus on SQL database queries.

Role: Disha is responsible for the user interface design and development, as well as managing the structure of the SQL queries and schema. Her role involves ensuring that the user inputs are correctly processed and converted into database-readable formats.

3. **Nikhil** – Full Stack Developer and NLP specialist with expertise in AI model training.

Role: Nikhil takes charge of backend development, overseeing the server-side logic and data flow. He is also responsible for training and fine-tuning AI models, ensuring they perform well in generating SQL queries from natural language inputs.

Collaborative Approach:

While each team member has specific responsibilities, the team works together on most elements of the project, including research, design, and development. This collaborative approach ensures that knowledge is shared, and all team members contribute to the core project objectives.

5.4.2 Management reporting and communication

1. **Internal Guide:** Prof. Rutuja Kulkarni provides academic and technical guidance, offering mentorship and ensuring the project aligns with academic goals.
2. **External Mentor:** Swapnil Bhusari from Veritas Technologies offers practical advice, linking the academic aspects of the project to real-world applications and industry standards.

Communication Approach:

- **Regular Meetings:** Weekly discussions via Google Meet or Zoom allow the team to review progress and make necessary adjustments.
- **Progress Updates:** Bi-weekly updates are provided to both the internal guide and external mentor.

CHAPTER 06

IMPLEMENTATION

6.1 OVERVIEW OF PROJECT MODULES

This project is designed to convert natural language SQL database configuration requests into structured JSON. It consists of the following modules:

1) Frontend Module:

- Provides a user interface for inputting natural language SQL descriptions.
- Displays the generated JSON output or error messages.
- Implements real-time communication with the backend using WebSocket (Socket.IO).
- Includes features like a loading spinner, session history, and error handling.

2) Backend Module:

- A Flask application that serves the frontend and handles WebSocket communication.
- Processes user inputs and generates JSON outputs using the LangChain framework.
- Includes error handling for invalid inputs and unexpected issues.

3) LLM Handler Module:

- Manages interaction with the Mistral AI model using the langchain-mistralai library.
- Sends prompts to the AI model and retrieves responses.

4) JSON Conversion Module:

- Defines the schema for the expected JSON output using LangChain's ResponseSchema.
- Contains the logic for converting natural language SQL descriptions into structured JSON.
- Ensures the output adheres to the specified schema and omits unspecified fields.

5) Prompt and Schema Module:

- Defines the system prompt and response schema for guiding the AI model.
- Includes examples and guidelines for generating valid JSON outputs.
- Provides format instructions for the AI model to follow.

6) Command-Line Interface Module:

- A standalone script for testing the JSON generation functionality via the command line.
- Allows users to input natural language SQL descriptions and view the generated JSON.

6.2 TOOLS AND TECHNOLOGIES USED

1) Frontend:

- HTML/CSS: For creating the user interface.
- JavaScript: For handling user interactions and WebSocket communication.
- Socket.IO: For real-time communication between the frontend and backend.

2) Backend:

- Flask: A lightweight web framework for serving the frontend and handling API requests.
- Flask-SocketIO: For enabling WebSocket communication in the backend.

3) AI and Natural Language Processing:

- LangChain: For managing prompt templates, structured output parsing, and interaction with the AI model.
- LangChain-MistralAI: For integrating with the Mistral AI model to generate responses.

4) WebSocket Communication:

- Socket.IO: For real-time, bidirectional communication between the frontend and backend.
- Python-SocketIO: For server-side WebSocket communication.
- Eventlet: For WebSocket support in Flask-SocketIO.

5) Other Libraries:

- Requests: For handling HTTP requests (used internally by some libraries).

6) Environment:

- Python 3.8+: The programming language used for the backend and AI integration.

- **Mistral AI API:** The AI service used for generating JSON outputs based on natural language inputs.

6.3 ALGORITHM DETAILS

6.3.1 Prompt Engineering Algorithm

- **Purpose:** Guide the AI model (Mistral AI) to generate structured JSON outputs based on natural language SQL descriptions.
- **How It Works:**
 - A system prompt is defined with examples, guidelines, and format instructions.
 - User input is combined with this prompt and sent to the AI model.
- **Role in Project:** Ensures outputs are structured, relevant, and aligned with user requirements.

6.3.2 Response Schema Validation Algorithm

- **Purpose:** Validate the structure of the AI-generated JSON output.
- **How It Works:**
 - ResponseSchema defines expected fields.
 - StructuredOutputParser ensures the AI output matches this structure.
- **Role in Project:** Prevents incomplete or invalid JSON outputs.

6.3.3 Natural Language Processing (NLP) Algorithm

- **Purpose:** Extract meaningful information from natural language SQL descriptions.
- **How It Works:**
 - The AI model uses NLP to identify key components like table names, column names, constraints, etc.
- **Role in Project:** Converts human-readable descriptions into machine-readable structured data.

6.3.4 WebSocket Communication Algorithm

- **Purpose:** Enable real-time communication between frontend and backend.

- **How It Works:**
 - Socket.IO handles the sending and receiving of messages between client and server.
- **Role in Project:** Provides a seamless and responsive user experience.

6.3.5 Error Handling Algorithm

- **Purpose:** Manage errors and provide meaningful feedback to users.
- **How It Works:**
 - Input validation and exception handling are used to identify issues and respond with appropriate messages.
- **Role in Project:** Ensures system robustness and improves user experience.

6.3.6 Session Management Algorithm

- **Purpose:** Maintain a history of user inputs and outputs during a session.
- **How It Works:**
 - The frontend keeps track of all interactions using a session history array.
- **Role in Project:** Enables users to review past queries and outputs.

6.3.7 JSON Serialization and Validation Algorithm

- **Purpose:** Ensure JSON outputs are well-formed and valid.
- **How It Works:**
 - Uses `json.dumps()` for serialization and schema validation for correctness.
- **Role in Project:** Prevents errors and ensures interoperability with other systems.

CHAPTER 07

SOFTWARE TESTING

7.1 TYPE OF TESTING

In this project, the following types of testing were performed to validate functionality and robustness:

- **Functional Testing:** To verify that the system correctly interprets natural language queries and generates valid JSON.
- **Schema Validation Testing:** To ensure generated JSON conforms to the defined schema structure.
- **Boundary Testing:** To test inputs near the limits of accepted formats (e.g., edge-case values in SQL).
- **Error Handling Testing:** To validate that the system correctly handles incorrect, ambiguous, or incomplete inputs.
- **Integration Testing:** To test the seamless interaction between the frontend, backend, LLM processing, and WebSocket communication.

7.2 TEST CASES AND TEST RESULTS

Table 7.2 Test Cases And Test Results

Test Case	Natural Language Input	Expected JSON Structure Generated	Result
1. Update Query	"Update the salary of employee from the employee table having id=1 to salary=60000"	Operation: UPDATE; Table: employee; Set: salary=60000; Condition: id=1	✓ Pass
2. Join with Grouping	"Show the number of orders per customer in the last year using Orders and Customers tables"	Operation: SELECT with JOIN; Columns: COUNT(order_id); Group By: customer_id; Filter: last 1 year	✓ Pass
3. Join with HAVING Clause	"List customers who placed more than 10 orders using Orders and Customers tables"	Operation: SELECT with JOIN; Condition: HAVING COUNT(order_id) > 10	✓ Pass
4. INSERT Query	"Insert a new product into the Products table with name 'Laptop', price 1000, and quantity 50."	Operation: INSERT; Table: Products; Fields: name, price, quantity; Values: Laptop, 1000, 50	✓ Pass
5. DELETE Query	"Delete all users who haven't logged in for over a year."	Operation: DELETE; Table: Users; Condition: last_login < [current_date - 1 year]	✓ Pass
6. TRANSACTION with ROLLBACK	"Rollback the current transaction if there's an error while inserting into the Payments table."	Operation: TRANSACTION; Action: INSERT into Payments; Error: Yes → Rollback triggered	✓ Pass

CHAPTER 08

RESULTS

8.1 OUTCOMES

The final implementation of the project successfully met all key objectives. The system was able to:

- Accurately interpret natural language SQL descriptions and convert them into structured JSON configurations.
- Handle a wide variety of SQL intents such as SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, and ALTER TABLE.
- Maintain real-time communication using WebSockets (Socket.IO) between frontend and backend for instant feedback.
- Provide a user-friendly interface with loading states, error handling, and session history.
- Validate all outputs against a predefined JSON schema using LangChain's structured output parsers.
- Ensure error robustness by gracefully handling incorrect or incomplete inputs and providing meaningful error messages.
- Seamlessly integrate with LLM (Mistral AI) via LangChain for prompt handling and dynamic response generation.
- Support both GUI-based interaction and command-line testing, ensuring development flexibility and easier debugging.

Overall, the system demonstrated high reliability, robustness, and effectiveness in bridging the gap between natural language and machine-readable database configurations.

8.2 SCREENSHOTS

Natural Language to JSON Converter

"update the salary of employee from the employee table having id=1 to salary=60000"

Send Input

Reset

Processing...

```
{
  "query_type": "UPDATE",
  "tables": [
    "employee"
  ],
  "columns": [
    "salary"
  ],
  "values": [
    {
      "salary": 60000
    }
  ],
  "joins": [],
  "conditions": [
    {
      "column": "id",
      "operator": "="
    }
  ]
}
```

Copy Output

Fig 8.2.1 Result Screenshot 01

Natural Language to JSON Converter

"update the salary of employee from the employee table having id=1 to salary=60000"

Send Input

Reset

```
{
  "values": [
    {
      "salary": 60000
    }
  ],
  "joins": [],
  "conditions": [
    {
      "column": "id",
      "operator": "=",
      "value": 1
    }
  ],
  "group_by": [],
  "having": [],
  "order_by": [],
  "limit": null,
  "alter_action": [],
  "transaction": []
}
```

Copy Output

Fig 8.2.2 Result Screenshot 02

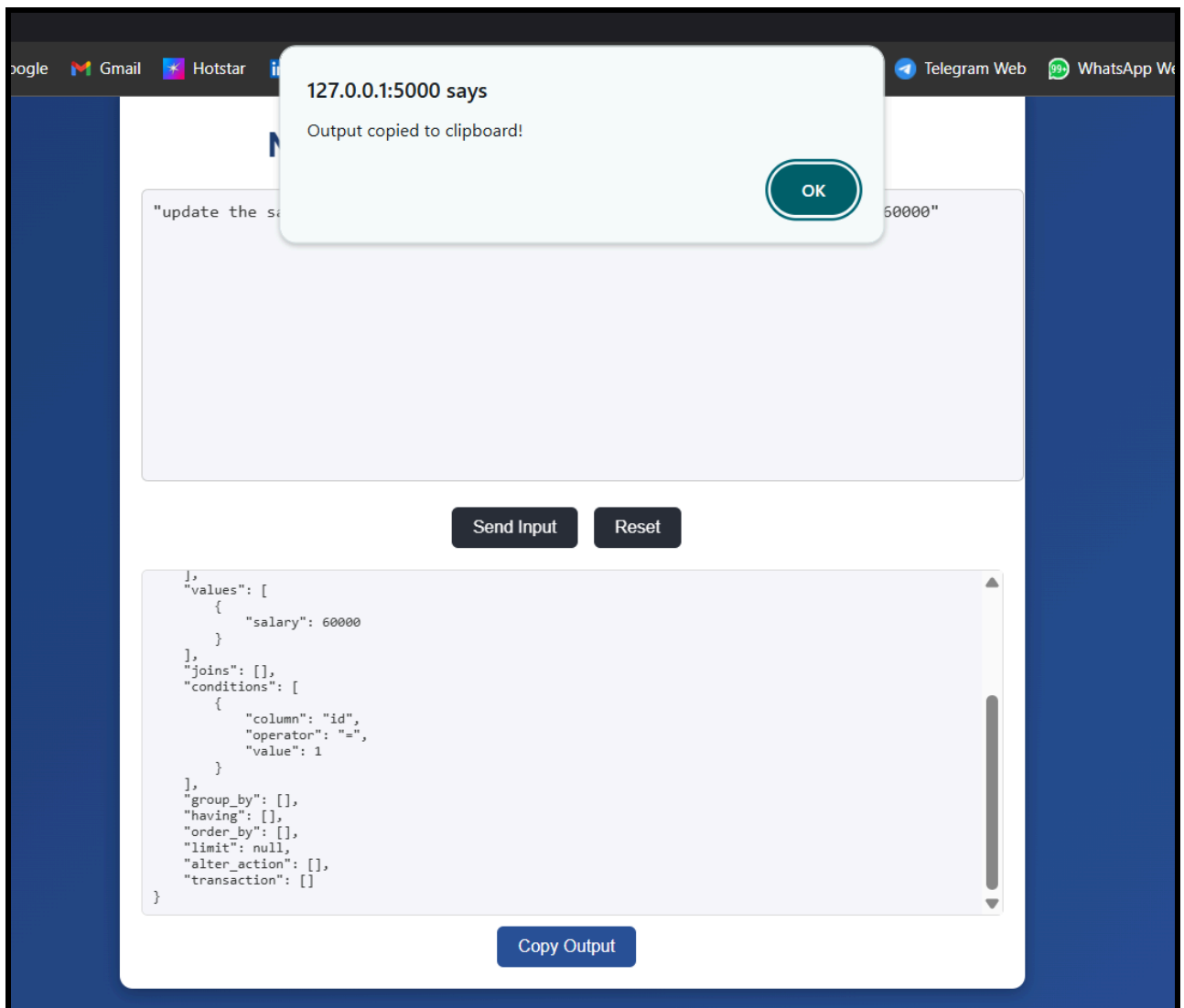


Fig 8.2.3 Result Screenshot 03

Natural Language to JSON Converter

Get product names and supplier names for all products supplied by suppliers located in Germany.

Send Input

Reset

Processing...

```
{
  "query_type": "UPDATE",
  "tables": [
    "employee"
  ],
  "columns": [
    "salary"
  ],
  "values": [
    {
      "salary": 60000
    }
  ],
  "joins": [],
  "conditions": [
    {
      "column": "id",
      "operator": "="
    }
  ]
}
```

Copy Output

Fig 8.2.4 Result Screenshot 04

Natural Language to JSON Converter

Get product names and supplier names for all products supplied by suppliers located in Germany.

Send Input

Reset

```
{
  "values": [],
  "joins": [
    {
      "type": "INNER",
      "condition": "products.supplier_id = suppliers.supplier_id"
    }
  ],
  "conditions": [
    {
      "column": "suppliers.country",
      "operator": "=",
      "value": "Germany"
    }
  ],
  "group_by": [],
  "having": [],
  "order_by": [],
  "limit": null,
  "alter_action": [],
  "transaction": []
}
```

Copy Output

Fig 8.2.5 Result Screenshot 05

Natural Language to JSON Converter

Get the total sales amount for each customer from the Orders and Customers tables, considering only orders placed in the last 6 months"

Send Input

Reset

Processing...

```
{
  "query_type": "SELECT",
  "tables": [
    "Orders",
    "Customers"
  ],
  "columns": [
    "Customers.CustomerID",
    "SUM(Orders.TotalAmount)"
  ],
  "values": [],
  "joins": [
    {
      "type": "INNER",
      "condition": "Orders.CustomerID = Customers.CustomerID"
    }
  ],
  "conditions": [
```

Copy Output

Fig 8.2.6 Result Screenshot 06

Natural Language to JSON Converter

Get the total sales amount for each customer from the Orders and Customers tables, considering only orders placed in the last 6 months"

Send Input

Reset

```
"SUM(Orders.TotalAmount)"
],
"values": [],
"joins": [
  {
    "type": "INNER",
    "condition": "Orders.CustomerID = Customers.CustomerID"
  }
],
"conditions": [
  "Orders.OrderDate >= DATE_SUB(CURRENT_DATE, INTERVAL 6 MONTH)"
],
"group_by": [
  "Customers.CustomerID"
],
"having": [],
"order_by": [],
"limit": null,
"alter_action": [],
"transaction": []
}
```

Copy Output

Fig 8.2.7 Result Screenshot 07

Natural Language to JSON Converter

Rollback the current transaction if there's an error while inserting into the Payments table.

Send Input

Reset

Processing...

```
{
  "query_type": "TRANSACTION",
  "tables": [
    "Payments"
  ],
  "columns": [],
  "values": [],
  "joins": [],
  "conditions": [],
  "group_by": [],
  "having": [],
  "order_by": [],
  "limit": null,
  "alter_action": [],
  "transaction": [
    "ROLLBACK"
  ]
}
```

Copy Output

Fig 8.2.8 Result Screenshot 08

CHAPTER 09

CONCLUSION

9.1 CONCLUSION

This project validates the ability of a Large Language Model (LLM) to bridge the gap between human language and structured query logic by converting natural language SQL configuration requests into JSON. The system efficiently interprets user inputs and delivers structured outputs with high consistency. Through modular design and real-time frontend-backend interaction, it ensures a smooth user experience and scalability. By lowering the entry barrier for SQL generation, the system enables faster decision-making, especially for business users without technical expertise.

9.2 FUTURE WORK

To enhance performance and usability, several areas can be explored in future iterations:

- **Ambiguity Resolution:** Improve the system's ability to detect and handle vague or incomplete inputs by adding clarification prompts or confidence-based outputs.
- **Multi-Intent Query Support:** Allow users to ask compound queries (e.g., "Create a table and insert values") in one prompt.
- **Context Awareness:** Maintain conversational context across multiple queries for dynamic query building.
- **Fine-Tuned LLM Models:** Fine-tune the LLM on industry-specific or organization-specific schema examples for better relevance.
- **Authentication & Role-Based Access:** Integrate user-level access control to ensure secure and permissioned query generation.
- **Execution & Preview Layer:** Add support for safe SQL preview and test-run execution in a sandboxed environment.

9.3 APPLICATIONS

This system holds strong potential for enterprise and corporate adoption across various functions:

- **Business Intelligence & Reporting:** Non-technical teams can query data directly using plain language, streamlining report generation.
- **Enterprise Resource Planning (ERP) Tools:** Employees can extract insights from ERP databases using natural prompts, saving time and reducing dependency on IT teams.
- **Customer Support Analytics:** Support teams can retrieve user activity or complaint data with ease by simply describing what they need.
- **Internal Data Portals:** Teams across HR, Sales, or Finance can use the system to query and analyze department-specific data without writing SQL.
- **IT Helpdesk Automation:** Automate backend data checks or audits triggered by helpdesk tickets using natural language instructions.

Appendix A: Problem Statement Feasibility Assessment

This section explores the feasibility of the problem statement using concepts from **computational complexity theory** (e.g., P, NP, NP-Complete, NP-Hard) and relevant mathematical modeling.

A.1 Problem Classification

The primary task in our project is converting **natural language SQL descriptions into structured JSON configuration objects**, which is inherently a **language translation and structured representation problem**.

From a computational complexity perspective:

Table A.1 Problem Statement Feasibility Assessment Table

Aspect	Classification	Reason
Natural Language Parsing	NP	Requires understanding all potential syntactic/semantic interpretations.
JSON Schema Validation	P	Linear validation using defined rules.
Prompt Interpretation Logic	NP-Complete	Mapping unstructured queries to structured schema resembles SAT-style decision problems.
Real-time Conversion + Error Handling	NP-Hard	Incorporates ambiguous inputs and incomplete information under time

A.2 Mathematical Modeling and Satisfiability Analysis

The problem can be broken down into **decision and optimization sub-problems**:

- **Decision Problem:** “Is there a valid JSON representation that satisfies all constraints of the prompt?”
 - Modeled as a **Satisfiability (SAT)** problem.
 - Variables: Detected SQL clauses, schema tags, query structure.

- Constraints: Must match expected JSON schema, complete all mandatory fields, and follow data type consistency.
- Feasibility: For limited schema and finite prompt length, SAT solvers can verify solutions in NP time.
- **Optimization Problem:** “What is the most accurate and minimal JSON output that best represents the input?”
 - Involves **heuristics** and **search-based AI**.
 - Not polynomial-time solvable in general, unless $P = NP$.

A.3 Theoretical Justification

- **NP-Complete Nature:** The mapping from natural language to structured schema involves a subset of **context-sensitive grammars**, which are **non-deterministic polynomial** problems.
- **Use of Modern Algebra:**
 - Set theory: Matching tokens to schema elements.
 - Function mappings: Prompt $\rightarrow$ JSON (many-to-one).
 - Boolean logic: For validating presence/absence of required fields using propositional logic rules.

Appendix B: Details of Paper Publication

- **Conference:**
The project was presented at the
7th International Conference on Recent Trends in Engineering and Technology (VISHWACON 2024),
organized by **Vishwakarma Institute of Technology (VIT), Pune**

Certificates:

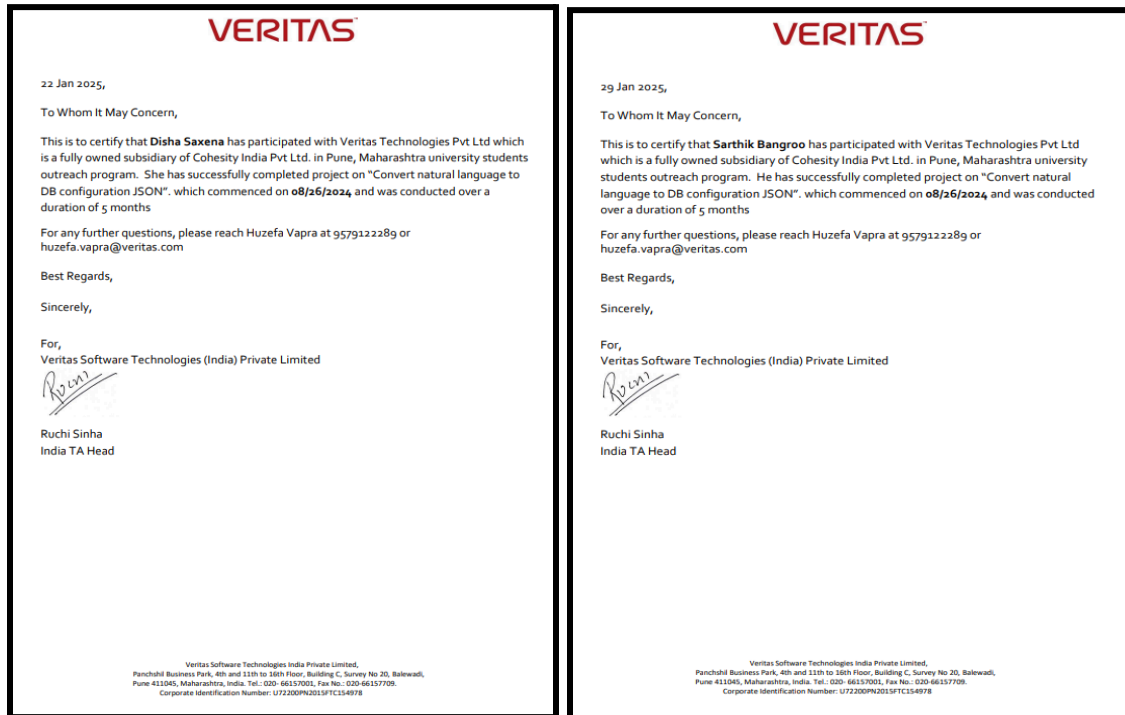


- **Publication (Survey Paper):**
The research paper derived from this project is currently **under review** for publication in the **Cureus Journal of Computer Science**.
- **Publication (Implementation paper):**
The research paper derived from this project is currently **submitted** for publication in the **International Conference on Business Intelligence, Computational Mathematics, and Data Analytics (IBCD 2025)**, D. Y. Patil Institute of Technology
- **Participation:**
Participated in Impetus and Concepts 24-25.

- **Sponsorship:**

This work is a **sponsored project under Veritas Technologies Ltd.**, as part of an industry-academia partnership aimed at exploring intelligent automation through natural language processing and database interfacing technologies.

Certificates:



Appendix C: Plagiarism Report Of Project Report

Document Information

Analyzed document	GRP_50_PROJECT_REPORT.docx.pdf (D210180264)
Submitted	2025-04-26 10:14:00 UTC+02:00
Submitted by	Uspawar
Submitter email	uspawar@pict.edu
Similarity	8%
Analysis address	uspawar.pict@analysis.arkund.com

Sources included in the report

W	URL: https://www.coursehero.com/file/83472522/PROJECT-REPORT-1pdf/ Fetched: 2025-04-26 10:14:00		1
W	URL: https://pravaraengg.org.in/naac/132/3_Proj_CO.pdf Fetched: 2025-04-26 10:15:00		1
W	URL: https://www.coursehero.com/file/70115371/UTMS-SRS-1doc/ Fetched: 2025-04-26 10:15:00		1
W	URL: https://WWW.coursehero.com/file/236179261/BlckBookReport-Finalpdf/ Fetched: 2025-04-26 10:15:00		2
W	URL: https://www.ijraset.com/best-journal/vibzey-an-ai-based-ios-music-app Fetched: 2025-04-26 10:15:00		1
W	URL: https://ojs.acad-pub.com/index.php/FES/issue/view/192 Fetched: 2025-04-26 10:15:00		1
W	URL: https://www.csauthors.net/syed-ahmad-chan-bukhari/ Fetched: 2025-04-26 10:15:00		3
W	URL: https://assets-eu.researchsquare.com/files/rs-70794/v1_covered.pdf?c=1631840983 Fetched: 2025-04-26 10:15:00		2
W	URL: https://www.isacaroma.it/wp-content/uploads/2019/11/Roma20191120AP.pdf Fetched: 2025-04-26 10:15:00		1
W	URL: https://www.coursehero.com/file/p5pl2rc/Medium-5-Schedule-impact-or-Barely-noticeable-degradation-in-qual-ity-Low-Impact/ Fetched: 2025-04-26 10:15:00		1
W	URL: http://dwl.kiev.ua/art/ssrn24.tm/ssrn-5000687.pdf Fetched: 2025-04-26 10:15:00		1

REFERENCES

1. Abdelkader Mekrache, Adlen Ksentini, Christos Verikoukis, et al.: “Intent-Based Management of Next-Generation Networks: an LLM-Centric Approach.” In *ACM Digital Library*, 2024, Vol. 38:36. 10.1109/MNET.2024.3420120.
2. Mustafa Man, Kamir Yusof: “Efficiency of JSON Approach for Data Extraction and Query Retrieval.” In *ResearchGate*, 2016, Vol. 4:214. 10.11591/ijeecs.v4.i1.pp203-214.
3. Syed Ahmad Chan Bukhari, Hafsa Shareef Dar, Muhammad Ikram Lali, Fazel Keshtkar: “Frameworks for Querying Databases Using Natural Language: A Literature Review - NLP-to-DB Querying Frameworks.” In *ResearchGate*, 2021, Vol. 17:38. 10.4018/IJDWM.2021040102.
4. Y. Liu, Y. Wang, Q. Yang: “How to Convert Natural Language to SQL Queries.” In *Upwork*, 2022, Vol. 1:30.
5. JR Oclarit, JV Compoc, D Jr Ancog, D Cardaña, MA Perin, RV Tejada: “SMARTPAWS: An Internet of Things (IoT) Pet Feeder for Dogs and Cats using Arduino Uno.” In *ResearchGate*, 2024, Vol. 99:7. 10.13140/RG.2.2.27220.00643.
6. Niranjan Gowda, Sinchana S. Noolee, T Vismaya, Vidyashree B S: “A Survey on Natural Language to SQL Query Generator.” In *IJRASET*, 2023, Vol. 11:321. 10.22214/ijraset.2023.51466.
7. Jingfeng Yang, Hongye Jin, Ruixiang Tang, et al.: “Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond.” In *Cornell University*, 2023, Vol. 2. 10.48550/arXiv.2304.13712.
8. Mikel Artetxe, Jingfei Du, Naman Goyal, et al.: “On the Role of Bidirectionality in Language Model Pre-Training.” In *Cornell University*, 2022, Vol. 2. 10.48550/arXiv.2205.11726.
9. Ilya Sutskever, Oriol Vinyals, Quoc V. Le: “Sequence to Sequence Learning with Neural Networks.” In *Cornell University*, 2014, Vol. 3. 10.48550/arXiv.1409.3215.
10. Hazrina Sofian, Nur Arzilawati Md Yunus, Rodina Ahmad: “Systematic Mapping: Artificial Intelligence Techniques in Software Engineering.” In *IEEE Explore*, 2022, Vol. 10:51040. 10.1109/ACCESS.2022.3174115.
11. John Chong Min Tan, Mehul Motani: “Large Language Model (LLM) as a System of Multiple Expert Agents: An Approach to Solve the Abstraction and Reasoning Corpus (ARC) Challenge.” In *Cornell University*, 2023, Vol. 1. 10.48550/arXiv.2310.05146.
12. Hendrik Strobelt, Albert Webson, Victor Sanh, Benjamin Hoover, Johanna Beyer, Hanspeter Pfister, Alexander M. Rush: “Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models.” In *Cornell University*, 2022, Vol. 1. 10.48550/arXiv.2208.0785.
13. H. Alawwad, E. Khan: “An Intelligent Database System using Natural Language Processing.” In *International Journal of Computers*, 2016, Vol. 1(1):120–126.
14. Albertodelazzari, Bachmanm, Ikwattro, Inserpio: “GraphAware Knowledge Platform.” In *GitHub*, 2014. Retrieved from <https://graphaware.com/>
15. R. Alexendar, P. Rukshan, S. Mahesan: “Natural Language Web Interface for Database (NLWIDB).” In *Proceedings of the Third International Symposium*, IEEE, 2013.
16. F. Basik, B. Hättasch, A. Ilkhechi, A. Usta, S. Ramaswamy, P. Utama, U. Cetintemel: “DBPal: A Learned NL-Interface for Databases.” In *Proceedings of the 2018 International Conference on Management of Data*, ACM, pp. 1765–1768, 2018. <https://doi.org/10.1145/3183713.3193562>.